



UD 9: Tarea 1.

Título: Persistencia de Objetos con ObjectDB.

Alumno: Kamal Majaiti El Majaiti.

Modulo: Programacion DAW.

Objetivo:

Desarrollar una aplicación en Java que permita almacenar y consultar objetos utilizando una base de datos orientada a objetos con ObjectDB.

La aplicación deberá:

- Conectarse a la base de datos.
- Persistir objetos.
- Recuperarlos posteriormente mediante consultas.

Programación
DAW

domingo, 26 de abril de 2026

Kamal Majaiti El Majaiti

Práctica de unidad 9: Persistencia de Objetos con ObjectDB

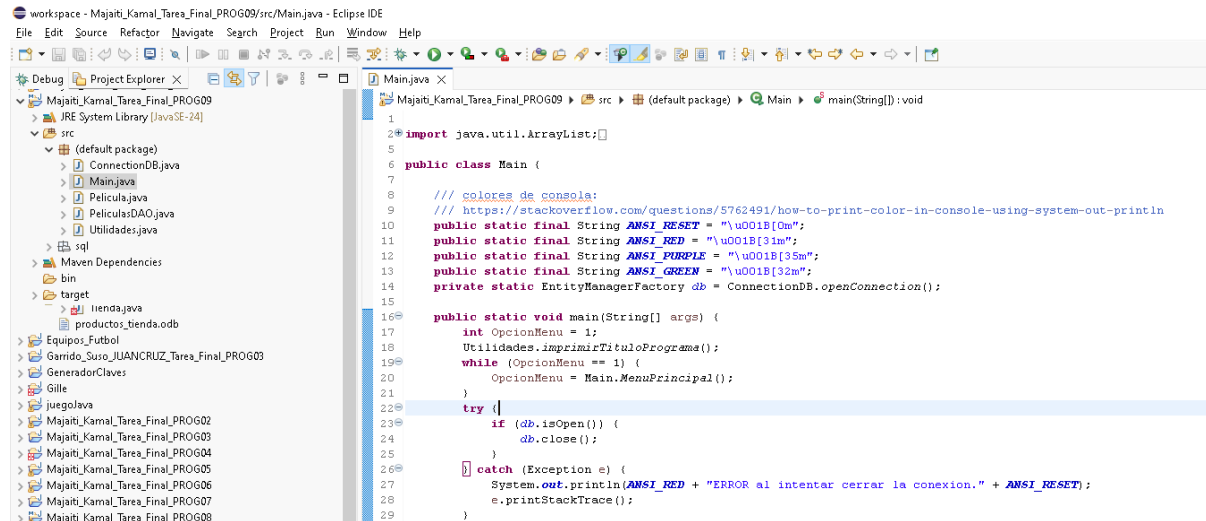
Contenido

1	La estructura del proyecto.	3
2	Revisión del código de la clase persistente del programa.	4
2.1	Fichero ConnectionDB.class.....	5
2.2	Fichero de clase de persistencia de datos, PeliculasDAO.class	6
2.2.1	Metodo para insertar una película en ObjectDB.	6
2.2.2	Metodo para comprobar si existe una película.	7
2.2.3	Metodo para actualizar la duración de una película.	8
2.2.4	Metodo para borrar una película.	9
2.2.5	Metodo para listar todas las películas de ObjectDB.	10
2.2.6	Metodo para listar los géneros distintos de las películas guardadas en ObjectDB.....	11
2.2.7	Metodo para listar las películas de un solo género.....	12
3	Resultados de Ejecucion y pruebas unitarias.	12
3.1	Menu inicial	12
4	Crear una película y listar todas las peliculas.	13
4.1	Listar películas por Genero.....	13
4.2	Eliminar una película guardada y listar todas las peliculas.....	14

Kamal Majaiti El Majaiti

Práctica de unidad 9: Persistencia de Objetos con ObjectDB

1 La estructura del proyecto.

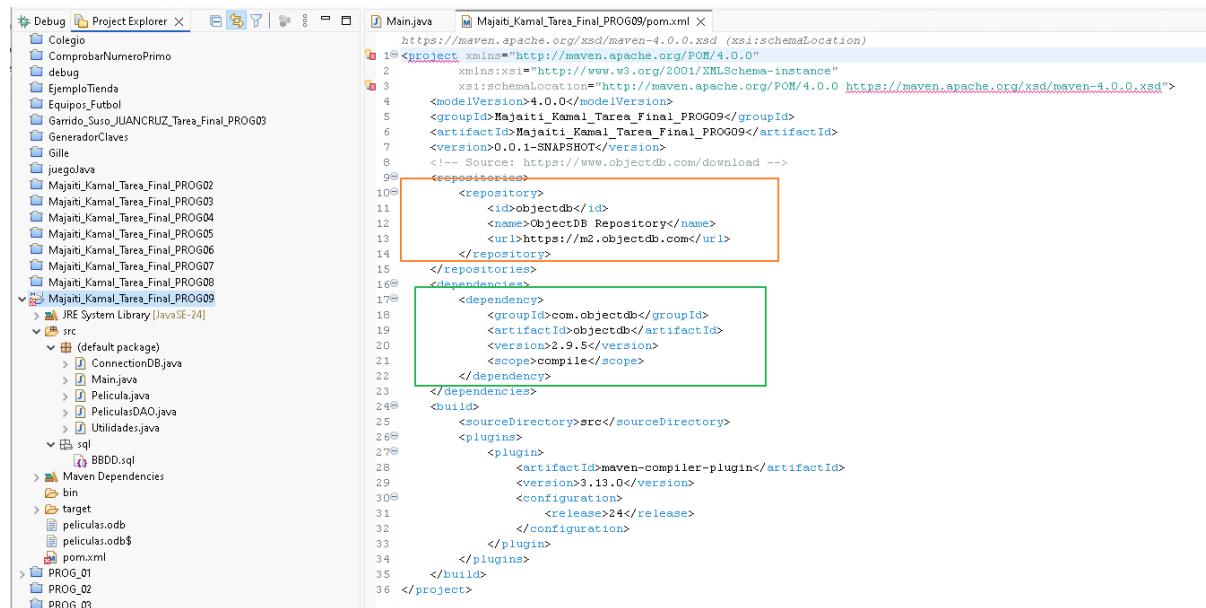


- **ConnectionDB.java:** Es la clase heredada de la unidad 8 y que abre la conexión contra MySQL/Mariadb y que he modificado para que use ObjectDB y lo que hace es abrir el fichero de persistencia de ObjectDB y devolver una clase que se utiliza para acceder a los datos y bloquear el fichero de persistencia que almacena los objetos durante la ejecución del programa.
- **Main.java:** Es el punto de entrada del programa, este contiene el método main y controla el menú principal y la lógica de interacción con el usuario. Este cuando elige una opción se encarga de imprimir el submenú y solicitar los datos al usuario.
- **Pelicula.java:** Es la clase que describe el objeto indicando sus atributos (título, año, director, etc.) y los métodos y constructores que definen el comportamiento del objeto instanciado.
- **PeliculasDAO.java:** Es el objeto de acceso a datos, heredado de la UD8 que he modificado para usar ObjectDB. Y en él se encuentra el código de la capa de persistencia y acceso a datos.
- **Utilidades.java** incluye múltiples métodos adicionales para obtener los datos que introduce el usuario por el teclado, validarlos y formatear diferentes mensajes de errores.

Kamal Majaiti El Majaiti

Práctica de unidad 9: Persistencia de Objetos con ObjectDB

Por ultimo se incluye un fichero pom.xml en el que se definen dos datos importantes, la dependencia de ObjectDB y el repositorio de ObjectDB. Los cuales he heredado de la unidad 8 y he modificado para usar ObjectDB usando la guía de “ObjectDB with Maven” disponible en: <https://www.ObjectDB.com/download>



2 Revisión del código de la clase persistente del programa.

Existen dos clases que tienen que ver con la persistencia del programa, ConnectionDB.class y PeliculasDAO.java.

En ambas clases, voy a colocar los comentarios indicando que hace cada clase y cada metodo.

Kamal Majaiti El Majaiti

Práctica de unidad 9: Persistencia de Objetos con ObjectDB

2.1 Fichero ConnectionDB.class

- Es una clase estática con 2 métodos uno abre el fichero de persistencia y el otro lo cierra.

```
import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;

public class ConnectionDB {

    // Fichero donde se almacena la persistencia del programa.
    public static final String obd="peliculas.odb";
    // Colores de la consola para mensajes informativos y de errores.
    public static final String ANSI_RESET = "\u001B[0m";
    public static final String ANSI_RED = "\u001B[31m";
    public static final String ANSI_PURPLE = "\u001B[35m";
    public static final String ANSI_GREEN = "\u001B[32m";
    // Metodo para abrir la conexion contra el fichero peliculas.odb y bloquearlo.
    public static EntityManagerFactory openConnection() {
        EntityManagerFactory emf = null;
        try {
            // Abrimos el fichero al que hace referencia el atributo estatico de tipo String obd.
            emf = Persistence.createEntityManagerFactory(ConnectionDB.obd);
            System.out.printf(ANSI_GREEN + "\nConectado a la BBDD: %s\n" + ANSI_RESET, ConnectionDB.obd );
        } catch (Exception e) {
            // Si da error pintamos el error en pantalla, solo daría error si existe una carpeta con el nombre.
            System.out.printf(ANSI_RED + "\nError al conectar a BBDD: %s \n"+ANSI_PURPLE+"Traza de
errores:\n"+ANSI_RESET, ConnectionDB.obd + ANSI_RESET);
            e.printStackTrace();
            Utilidades.pausarInteracion();
            System.out.println("Saliendo del programa, no se puede conectar a BBDD.");
            System.exit(1);
        }
        return emf;
    };

    // Metodo para cerrar la conexion contra el fichero peliculas.odb y desbloquearlo.
    public static void closeConnection(EntityManagerFactory conexion) {
        try {
            // Si está abierto el fichero lo cerramos.
            if (conexion != null && conexion.isOpen()) {
                conexion.close();
            }
        } catch (Exception e) {
            // Si no logramos cerrar el fichero damos error indicando la causa y la pila de llamadas.
            System.out.println("Error al cerrar la conexion de BBDD.");
            e.printStackTrace();
            Utilidades.pausarInteracion();
            System.out.println("Saliendo del programa, no se puede cerrar la conexion de BBDD.");
            System.exit(1);
        }
    };
}
```

Kamal Majaiti El Majaiti

Práctica de unidad 9: Persistencia de Objetos con ObjectDB

2.2 Fichero de clase de persistencia de datos, PeliculasDAO.class

En esta clase estática se incluyen todos los métodos para mantener coherente la capa de persistencia de datos, sus metodos nos permiten realizar las operaciones crear, eliminar, actualizar y recuperar objetos almacenados el ObjectDB en funcion de las necesidades del programa y la interacción del usuario.

2.2.1 Metodo para insertar una película en ObjectDB.

```
public static void insertarPelicula(Pelicula PeliculaNueva, EntityManagerFactory conexion) {  
  
    // Al metodo se le pasa la conexion o puntero de persistencia de EntityManager  
    // de ObjectDB ya instanciado al abrir el programa y se reutiliza.  
    // Comprobamos que la pelicula no tiene nulos.  
    if (PeliculaNueva == null || PeliculaNueva.getTitulo() == null || PeliculaNueva.getGenero() == null) {  
        System.out.println(ANSI_RED + "El objeto pelicula que se intenta insertar no es valido." + ANSI_RESET);  
        return;  
    }  
  
    try {  
        EntityManager em = conexion.createEntityManager(); // Creamos una nueva sesion de BBDD.  
        em.getTransaction().begin(); // Iniciamos una nueva transacción.  
        em.persist(PeliculaNueva); // Guardamos los datos del Objeto Pelicula.  
        em.getTransaction().commit(); // Salvamos los cambios en el fichero.  
        System.out.println(ANSI_GREEN + "Película agregada correctamente." + ANSI_RESET);  
    } catch (Exception e) {  
        // Si da error pintamos la causa.  
        System.out.println(ANSI_RED + "Error al agregar la Película a BBDD" + ANSI_RESET);  
        e.printStackTrace();  
    }  
}
```

Kamal Majaiti El Majaiti

Práctica de unidad 9: Persistencia de Objetos con ObjectDB

2.2.2 Metodo para comprobar si existe una película.

Este metodo es usado para comprobar si existe una película antes de añadirla, al borrarla o para buscarla, usando la entidad "Titulo" como clave principal.

```
public static boolean existePelícula(String Titulo, EntityManagerFactory conexion) {
    EntityManager em = conexion.createEntityManager();
    try {
        // https://stackoverflow.com/questions/17136162/ObjectDB-select-where-query
        // Me he basado en el ejemplo de ConsultaAlimentacion.java en vez de usar un
        // ArrayList he usado count. Por lo visto tengo que usar Long y no Integer me
        // daba error.
        // Consulta que comprueba si un objeto con un título ya existe.
        // Se indica el título y se obtiene el resultado.
        Long resultadoConsultaInt = em
            .createQuery("SELECT COUNT(p) FROM Película p WHERE p.Titulo = :Titulo",
Long.class)
            .setParameter("Titulo", Titulo).getSingleResult();

        em.close();
        // Devuelve true si es mayor que 0 indicando que ya existe la película.
        return resultadoConsultaInt > 0 ? true : false;
    } catch (Exception e) {
        // Si da error se lo mostramos al usuario y pintamos la pila de llamadas para
        // determinar la causa.
        System.out.println(ANSI_RED + "Error al comprobar la Película a BBDD" +
ANSI_RESET);
        e.printStackTrace();
        return false;
    }
}
```

Kamal Majaiti El Majaiti

Práctica de unidad 9: Persistencia de Objetos con ObjectDB

2.2.3 Metodo para actualizar la duración de una película.

```
public static void actualizarDuracion(String titulo, int duracion, EntityManagerFactory
conexion) {
    // Comprobamos si existe la pelicula, si no existe volvemos del metodo sin hacer nada.
    if (!existePelicula(titulo, conexion)) {
        System.out.printf(ANSI_RED
            + "Error al intentar actualizar la Pelicula con titulo \"%s\" en la BBDD,
ya que no existe.\n"
            + ANSI_RESET, titulo);
        return;
    }
    // Manual de como actualizar en ObjectDB.
    // https://www.ObjectDB.com/java/jpa/persistence/update
    // https://www.ObjectDB.com/java/jpa/entity/id#entity_identification
    // Creamos una nueva sesion contra BBDD.
    EntityManager em = conexion.createEntityManager();
    try {
        // Iniciamos una nueva TX.
        em.getTransaction().begin();
        // Recuperamos el objeto película de ObjectDB.
        Pelicula PeliculaActualizar = em.find(Pelicula.class, titulo);
        // Actualizamos la película obtenida con su metodo.
        PeliculaActualizar.setDuracion_minutos(duracion);
        // Guardamos los cambios.
        em.getTransaction().commit();
        // Terminamos la sesion.
        em.close();

    } catch (Exception e) {
        // Si da error pintamos el error y la pila de llamadas.
        System.out.println(ANSI_RED + "Error al comprobar la Pelicula a BBDD" +
ANSI_RESET);
        e.printStackTrace();
    }
}
```


Kamal Majaiti El Majaiti

Práctica de unidad 9: Persistencia de Objetos con ObjectDB

2.2.4 Metodo para borrar una película.

```
public static void eliminarPelicula(String titulo, EntityManagerFactory
conexion) {
    // Comprobamos si la película existe, si no existe salimos del
metodo.
    if (!existePelicula(titulo, conexion)) {
        System.out.printf(
            ANSI_RED + "Error al intentar borrar la Película con
titulo \"%s\" de la BBDD, porque no existe.\n"
            + ANSI_RESET,
            titulo);
        return;
    }
    // Metodo borrar: https://ObjectDB.com/java/jpa/persistence/delete
    // Creamos una sesion nueva.
    EntityManager em = conexion.createEntityManager();
    try {
        // Iniciamos la TX.
        em.getTransaction().begin();
        // Recuperamos el objeto película de ObjectDB,
        // Luego lo borramos, guardamos y cerramos la sesion.
        Pelicula PeliculaEliminar = em.find(Pelicula.class, titulo);
        em.remove(PeliculaEliminar);
        em.getTransaction().commit();
        em.close();

    } catch (Exception e) {
        // Si da error, lo mostramos la usuario y pintamos la pila de llamadas.
        System.out.println(ANSI_RED + "Error al comprobar la Película a
BBDD" + ANSI_RESET);
        e.printStackTrace();
    }
};
```

Kamal Majaiti El Majaiti

Práctica de unidad 9: Persistencia de Objetos con ObjectDB

2.2.5 Metodo para listar todas las películas de ObjectDB.

```
public static ArrayList<Película> listarTodasLasPelículas(EntityManagerFactory conexion) {  
    // Creamos una nueva sesion.  
    EntityManager em = conexion.createEntityManager();  
    // Creamos una lista vacía de objetos película para guardar las películas obtenidas desde objectDB.  
    List<Película> resultadoConsulta = null;  
    try {  
        // Iniciamos una nueva transacción  
        em.getTransaction().begin();  
        // Recuperamos todas las películas de Objectdb.  
        resultadoConsulta = em.createQuery("SELECT p FROM Película p", Película.class).getResultList();  
        // Cerramos la conexion  
        em.close();  
        // Casteamos List a ArrayList y devolvemos el resultado.  
        // https://stackoverflow.com/questions/5134466/how-to-cast-arraylist-from-list  
        return new ArrayList<>(resultadoConsulta);  
    } catch (Exception e) {  
        // En caso de error, pintamos el mensaje y la pila de llamadas.  
        System.out.println(ANSI_RED + "Error al obtener la Película a BBDD" + ANSI_RESET);  
        e.printStackTrace();  
    }  
    // Casteamos List a ArrayList y devolvemos el resultado.  
    // https://stackoverflow.com/questions/5134466/how-to-cast-arraylist-from-list  
    return new ArrayList<>(resultadoConsulta);  
};
```

Kamal Majaiti El Majaiti

Práctica de unidad 9: Persistencia de Objetos con ObjectDB

2.2.6 Metodo para listar los géneros distintos de las películas guardadas en ObjectDB.

```
public static ArrayList<String> obtenerGeneros(EntityManagerFactory conexion) {  
    // Creamos una nueva sesion.  
    EntityManager em = conexion.createEntityManager();  
    // Creamos una lista vacia para guardar las categorias que obtendremos más adelante.  
    List<String> resultadoConsulta = null;  
    try {  
        // Iniciamos una nueva TX, aunque para leer no hace falta...pero bueno.  
        em.getTransaction().begin();  
        // Recuperamos todas las categorias unicas de peliculas de Objectdb.  
        resultadoConsulta = em.createQuery("SELECT distinct p.genero FROM Pelicula p", String.class)  
            .getResultList();  
        em.close();  
        // Devolvemos las categorias obtenidas casteando a ArrayList.  
        // https://stackoverflow.com/questions/5134466/how-to-cast-arraylist-from-list  
        return new ArrayList<>(resultadoConsulta);  
    } catch (Exception e) {  
        System.out.println(ANSI_RED + "Error al obtener los géneros de las películas desde BBDD" +  
ANSI_RESET);  
        e.printStackTrace();  
        return new ArrayList<>();  
    }  
};
```

Kamal Majaiti El Majaiti

Práctica de unidad 9: Persistencia de Objetos con ObjectDB

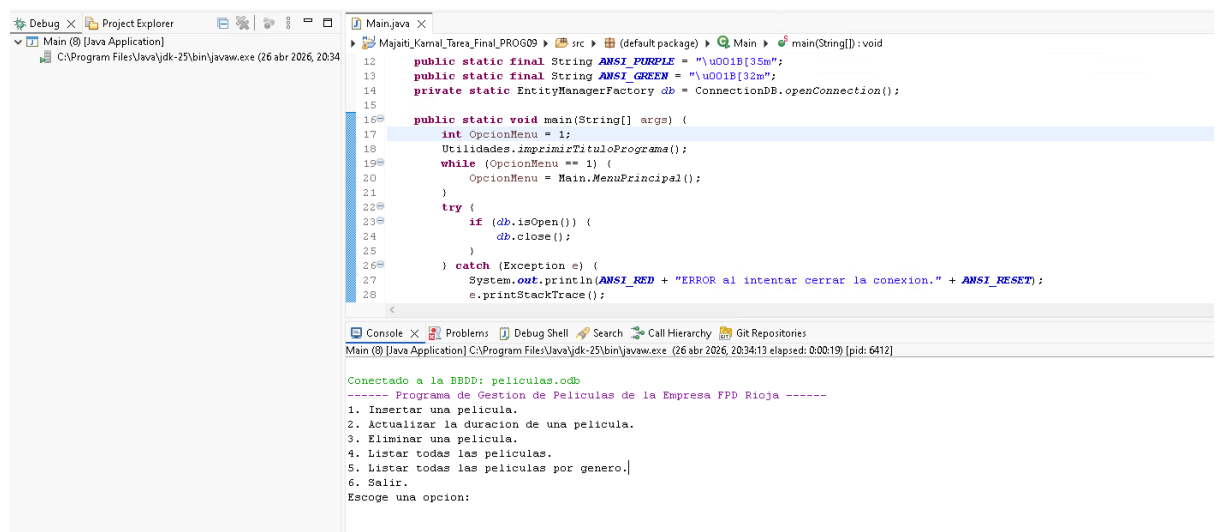
2.2.7 Metodo para listar las películas de un solo género.

```
public static ArrayList<Película> listarporGenero(String Genero, EntityManagerFactory conexion) {
    // Creamos una nueva sesion.
    EntityManager em = conexion.createEntityManager();
    // Creamos una lista donde guardar los objetos película que pertenecen al género indicado.
    List<Película> resultadoConsulta = null;
    try {
        // Iniciamos una nueva TX aunque no hace falta en lectura.
        em.getTransaction().begin();
        // Recuperamos todas los objetos de película de Objectdb que pertenecen a ese género.
        resultadoConsulta = em.createQuery("SELECT p FROM Película p WHERE p.genero = :genero", Película.class)
            .setParameter("genero", Genero).getResultList();
        // Cerramos la sesion.
        em.close();
        // Devolvemos el resultado pedido.
        return new ArrayList<>(resultadoConsulta);
    } catch (Exception e) {
        // Indicamos el error y pintamos la pila de llamadas.
        System.out.println(ANSI_RED + "Error al obtener la Pelicula a BBDD" + ANSI_RESET);
        e.printStackTrace();
    }
    // Devolvemos un listado vacio...
    return new ArrayList<>(resultadoConsulta);
}
};
```

3 Resultados de Ejecucion y pruebas unitarias.

3.1 Menu inicial

Menu inicial: Se observa que la cadena de BBDD es un fichero de persistencia y a continuación se muestra el menu.



```

Main.java
12 public static final String ANSI_PURPLE = "\u001B[35m";
13 public static final String ANSI_GREEN = "\u001B[32m";
14 private static EntityManagerFactory db = ConnectionDB.openConnection();
15
16 public static void main(String[] args) {
17     int OpcionMenu = 1;
18     Utilidades.imprimirTituloPrograma();
19     while (OpcionMenu == 1) {
20         OpcionMenu = Main.MenuPrincipal();
21     }
22     try {
23         if (db.isOpen()) {
24             db.close();
25         }
26     } catch (Exception e) {
27         System.out.println(ANSI_RED + "ERROR al intentar cerrar la conexion." + ANSI_RESET);
28         e.printStackTrace();
29     }
30 }

```

```

Conectado a la BBDD: peliculas.ojb
----- Programa de Gestion de Peliculas de la Empresa FPD Rioja -----
1. Insertar una pelicula.
2. Actualizar la duracion de una pelicula.
3. Eliminar una pelicula.
4. Listar todas las peliculas.
5. Listar todas las peliculas por genero.
6. Salir.
Escoge una opcion:

```

Práctica de unidad 9: Persistencia de Objetos con ObjectDB

- Se crea la película Titanic y a continuación se lista usando la opción 4, aparece la N° 3.

Kamal Majaiti El Majaiti

Práctica de unidad 9: Persistencia de Objetos con ObjectDB

3.4 Eliminar una película guardada y listar todas las películas.

- Se procede a eliminar una película (opcion 3) que no existe primero, luego se elimina la película Titanic y por ultimo se listan las películas guardadas.

```
----- Programa de Gestion de Peliculas de la Empresa FPD Rioja -----
----- Listar pelicula guardadas. -----
---- Pelicula N° 1 ----
Titulo: The matrix
Director: Kamal
Genero: Ciencia Ficción
Año de estreno: 2000
Duracion: 177 Minutos.
---- Pelicula N° 2 ----
Titulo: Titanic
Director: James Cameron
Genero: Drama
Año de estreno: 1997
Duracion: 190 Minutos.
Pulsa intro para continuar....
1. Insertar una pelicula.
2. Actualizar la duracion de una pelicula.
3. Eliminar una pelicula.
4. Listar todas las peliculas.
5. Listar todas las peliculas por genero.
6. Salir.
Escoge una opcion: 3
----- Programa de Gestion de Peliculas de la Empresa FPD Rioja -----
----- Eliminar una pelicula guardada. -----
Dime el titulo de pelicula: fff
ERROR: No existe una pelicula con este titulo.
Dime el titulo de pelicula que exista:
El titulo de pelicula que exista introducido no es valido.
Dime un titulo de pelicula que exista valido [a-zA-Z0-9]: Titanic
Pulsa intro para continuar....
1. Insertar una pelicula.
2. Actualizar la duracion de una pelicula.
3. Eliminar una pelicula.
4. Listar todas las peliculas.
5. Listar todas las peliculas por genero.
6. Salir.
Escoge una opcion: 4
|----- Programa de Gestion de Peliculas de la Empresa FPD Rioja -----
----- Listar pelicula guardadas. -----
---- Pelicula N° 1 ----
Titulo: The matrix
Director: Kamal
Genero: Ciencia Ficción
Año de estreno: 2000
Duracion: 177 Minutos.
Pulsa intro para continuar....
```

Kamal Majaiti El Majaiti

Práctica de unidad 9: Persistencia de Objetos con ObjectDB
